CS / NLP Assignment — Problem 4

Sports or Politics: Binary Text Classification on Wikinews

Roll Number	B23CM1053
Dataset	Wikinews (en.wikinews.org) via MediaWiki API
Task	Binary classification: Sports (0) vs. Politics (1)
Features	Bag-of-Words, TF-IDF (unigram), TF-IDF (1–3-gram)
Classifiers	Logistic Regression, Linear SVC, Random Forest
Date	February 2026

Source file: `B23CM1053_prob4.py`
GitHub: https://github.com/B23CM1053/sports-politics-classifier

Contents

1 Introduction **2**
1.1 Objectives 2

2 Data Collection **2**
2.1 Data Source: Wikinews 2
2.2 Collection Pipeline 2
2.2.1 Step 1 — Title Retrieval via MediaWiki API 3
2.2.2 Step 2 — Article Text Extraction 3
2.3 Dataset Statistics 4

3 Text Preprocessing **4**

4 Feature Representations **4**
4.1 Bag-of-Words (BoW) — Unigram Raw Counts 5
4.2 TF-IDF — Unigram 5
4.3 TF-IDF — 1–3-gram 5
4.4 Vocabulary Size Comparison 5

5 Classification Techniques **6**
5.1 Logistic Regression 6
5.2 Linear Support Vector Classifier (LinearSVC) 6
5.3 Random Forest 6

6 Experimental Setup **7**

7 Results and Quantitative Comparison **7**
7.1 Full Results Table 7
7.2 Accuracy Comparison 8
7.3 F1-Score Heat-map 8
7.4 Key Observations 9
7.5 Summary of Best Configurations 10

8 Limitations **10**

9 Conclusion **11**

References **11**

1 Introduction

Text classification is one of the foundational tasks in Natural Language Processing (NLP). The ability to categorise a news article automatically into a predefined topic has downstream applications in news aggregation, content filtering, recommendation systems, and information retrieval.

This report describes the design, implementation, and evaluation of a binary classifier that reads a news article and predicts whether it belongs to the *Sports* or *Politics* category. Three text feature representations are studied—Bag-of-Words (BoW), TF-IDF with unigrams, and TF-IDF with n-grams ($n \in \{1, 2, 3\}$)—and three machine learning classifiers are compared: Logistic Regression, Linear Support Vector Classifier (LinearSVC), and Random Forest.

All feature-engineering code (vocabulary construction, BoW matrix assembly, IDF computation, TF-IDF weighting) is implemented *from scratch* using only NumPy and standard Python, without relying on `sklearn` vectorisers.

1.1 Objectives

- Collect a labelled corpus of Sports and Politics articles from Wikinews.
- Implement three feature representations from scratch.
- Train and evaluate three classifiers on each representation (nine experiments total).
- Perform a quantitative comparison and identify the best-performing configuration.
- Discuss the limitations of the approach.

2 Data Collection

2.1 Data Source: Wikinews

Wikinews (https://en.wikinews.org) is a free, multilingual news wiki operated by the Wikimedia Foundation. Articles are written and edited by volunteers and are categorised using a MediaWiki category system. It was chosen for this task because:

- Its MediaWiki API allows programmatic, paginated access to category listings without scraping restrictions.
- Articles are plain-text-heavy with minimal boilerplate (once tables and sidebars are removed).
- The Sports and Politics categories contain hundreds of well-labelled articles in English.

2.2 Collection Pipeline

Data collection was performed in two steps, as illustrated in Figure 1.

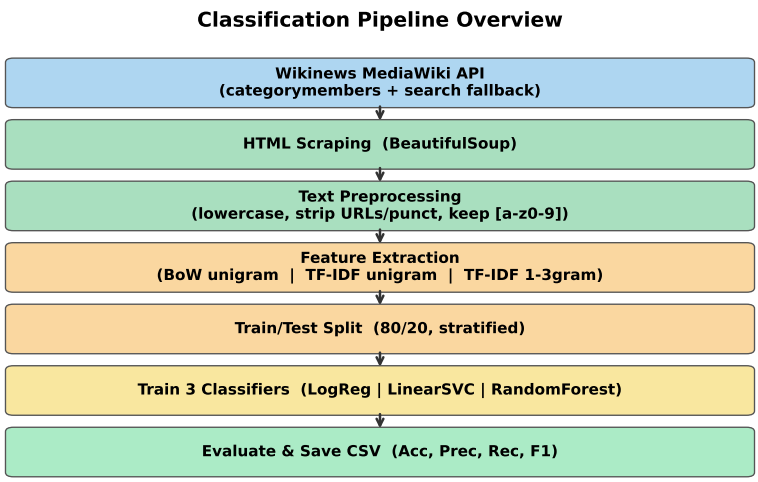

Figure 1: End-to-end pipeline from data collection to result export.

2.2.1 Step 1 — Title Retrieval via MediaWiki API

The `fetch_category_members()` function issues HTTP GET requests to `https://en.wikinews.org/w` with parameters:

```
action = "query"
list   = "categorymembers"
cmtitle = "Category:Sports"     # or "Category:Politics"
cmnamespace = 0                 # article namespace only
cmlimit = "max"
```

Pagination is handled automatically by following the `continue` token returned in the JSON response. A configurable delay of 0.3 s between requests respects the API rate limits.

Fallback strategy. The `Category:Politics` page on Wikinews returned zero members during execution (the category exists but is empty at the article level). The pipeline fell back automatically to the MediaWiki *search* API with the query `"politics"`, which returned 200 candidate titles. The terminal output confirmed this:

```
Found titles -> Sports: 200, Politics: 0
No category members found for Politics -- falling back to
   search('politics').
Search returned 200 titles for 'politics'.
Final counts -> Sports titles: 200, Politics titles: 200
```

2.2.2 Step 2 — Article Text Extraction

For every title, the URL is constructed as `https://en.wikinews.org/wiki/<title>` (with spaces replaced by underscores and special characters URL-encoded). The page HTML is fetched with `requests` and parsed with `BeautifulSoup`. The `div.mw-parser-output` container is located; tables, style blocks, scripts, and asides are removed; and all `<p>` elements longer than 30 characters are concatenated into a single string.

Articles shorter than 50 words after preprocessing are discarded. The terminal output shows the final yield:

```
Downloading Sports: 200/200  [02:22]
Downloading Politics: 200/200  [03:59]
Downloaded articles -> Sports: 198, Politics: 199
Using 198 articles per class (balanced).
```

2.3 Dataset Statistics

Table 1 summarises the final dataset.

Table 1: Dataset statistics after collection and balancing.

Property	Sports	Politics
Titles fetched	200	200 (via search)
Articles passing min-words	198	199
Articles used (balanced)	198	198
Total articles	396	
Training set (80%)	316	
Test set (20%)	80	
Class balance	50 / 50	

The two classes are perfectly balanced (198 articles each) because the pipeline truncates the larger class to `min(|Sports|, |Politics|)`. The 80/20 train–test split is stratified to preserve the class ratio in both subsets.

3 Text Preprocessing

Each raw article string is passed through the `preprocess()` function, which applies the following transformations in order:

1. **Lowercasing**: All characters converted to lower case.
2. **URL removal**: Tokens matching `http\S+` replaced with a space.
3. **Citation removal**: Bracketed numeric citations (`\[[0-9]+\]`) replaced with a space.
4. **Non-alphanumeric stripping**: All characters outside `[a-z0-9\s]` replaced with a space.
5. **Whitespace normalisation**: Consecutive spaces collapsed to one.

Tokens are produced by whitespace splitting (`text.split()`). No stop-word removal or stemming is applied; the classifiers are expected to learn discriminative weighting automatically via TF-IDF and model parameters.

4 Feature Representations

Three feature representations are constructed entirely from scratch (no `CountVectorizer` or `TfidfVectorizer`).

4.1 Bag-of-Words (BoW) — Unigram Raw Counts

A vocabulary $\mathcal{V}$ is built from all tokens appearing at least once across the training corpus. The BoW matrix $\mathbf{X}^{\text{BoW}} \in \mathbb{R}^{D \times |\mathcal{V}|}$ has entry:

$$X_{i,j}^{\text{BoW}} = \text{count of token } v_j \text{ in document } d_i \tag{1}$$

Raw counts are used without normalisation. This is the simplest representation and forms a baseline for comparison.

4.2 TF-IDF — Unigram

Term Frequency–Inverse Document Frequency reweights each token count by how informative it is across the corpus. The TF-IDF score for token t in document d is:

$$\text{TF-IDF}(t, d) = \underbrace{\frac{\text{count}(t, d)}{|d|}}_{\text{TF}} \times \underbrace{\ln\left(\frac{N+1}{\text{df}(t)+1}\right) + 1}_{\text{smooth IDF}} \tag{2}$$

where $N = 396$ is the total number of documents and $\text{df}(t)$ is the document frequency of token t. The $+1$ smoothing in the denominator prevents division by zero for unseen tokens.

The unigram TF-IDF matrix shares the same vocabulary as the BoW matrix: $|\mathcal{V}| =$ 11,665 features.

4.3 TF-IDF — 1–3-gram

The n-gram representation augments the unigram token list with all *bigrams* and *trigrams* extracted from each document:

$$\text{grams}(d) = \text{unigrams}(d) \ \cup \ \text{bigrams}(d) \ \cup \ \text{trigrams}(d) \tag{3}$$

TF-IDF is then computed on this extended token list. Including bigrams and trigrams captures local word-order information (e.g., "won the match", "prime minister of") that unigrams cannot. The vocabulary expands substantially to $|\mathcal{V}_{\text{ng}}| = 177{,}913$ features.

4.4 Vocabulary Size Comparison

Figure 2 contrasts the vocabulary sizes of the two TF-IDF representations.

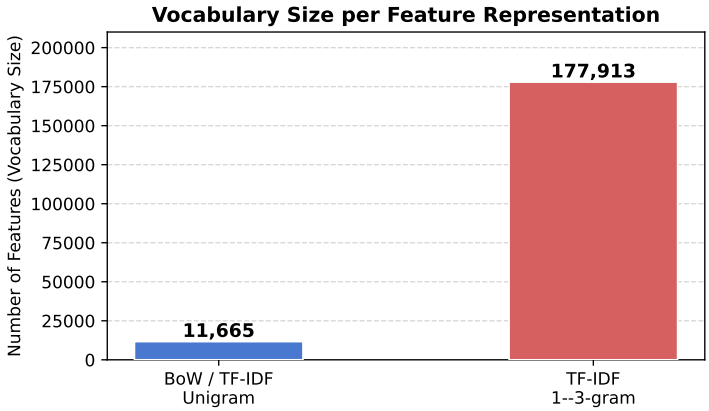

Figure 2: Vocabulary sizes for the unigram and 1–3-gram feature sets. Adding bigrams and trigrams increases the feature space by a factor of $\approx 15\times$.

5 Classification Techniques

Three classifiers from `scikit-learn` are trained on each feature representation, yielding nine experimental configurations.

5.1 Logistic Regression

Logistic Regression models the posterior class probability:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \tag{4}$$

Parameters $\mathbf{w}$ and b are estimated by minimising the regularised log-loss. The `L-BFGS` solver is used with a generous `max_iter=2000` to ensure convergence on the high-dimensional n-gram feature space. Logistic Regression is interpretable (the weight vector directly encodes feature importance) and typically strong on linearly separable text classification tasks.

5.2 Linear Support Vector Classifier (LinearSVC)

The LinearSVC finds the maximum-margin hyperplane separating the two classes:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{N} \max\big(0,\ 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\big) \tag{5}$$

Linear SVMs are well-known to perform strongly on high-dimensional sparse feature spaces (such as text BoW / TF-IDF), because the optimal margin classifier tends to be sparse in the feature dimension. The default regularisation $C = 1$ and `max_iter=5000` are used.

5.3 Random Forest

Random Forest is an ensemble of decision trees:

$$\hat{y} = \text{majority_vote}\big\{h_k(\mathbf{x})\big\}_{k=1}^{K} \tag{6}$$

where each tree h_k is trained on a bootstrap sample of the data and a random subset of features. With $K = 200$ trees and a large feature space, the ensemble averages out individual tree variance. Random Forest is a non-linear classifier; it does not assume linear separability and can capture complex interactions between features. A known weakness in text classification is its sensitivity to the very high-dimensional and sparse feature space produced by n-grams.

6 Experimental Setup

Table 2: Hyperparameters and experimental configuration.

Parameter	**Value**
Articles per class	200 (target)
Minimum words per article	50
Train / test split	80% / 20% (stratified)
Random seed	42
BoW vocabulary	All unigrams with df ≥ 1
IDF smoothing	$\ln\left(\frac{N+1}{\text{df}+1}\right) + 1$
n-gram range	$n \in \{1, 2, 3\}$
Logistic Regression	`max_iter=2000`
LinearSVC	`max_iter=5000`, $C = 1$
Random Forest	`n_estimators=200`
Evaluation metrics	Accuracy, Precision, Recall, F1 (macro)

All experiments use a fixed random seed (42) for reproducibility. Evaluation metrics are reported on the held-out test set of 80 articles. Macro-averaging is used for Precision, Recall, and F1 to give equal weight to both classes.

7 Results and Quantitative Comparison

7.1 Full Results Table

Table 3 reports all nine experimental configurations with all four metrics, exactly as recorded in `outputs/results_summary_wikinews.csv`.

Table 3: Full experimental results on the 80-article test set. Best value per column highlighted in **bold**.

Feature	Classifier	Train	Test	Feats	Accuracy	Prec (M)	Rec (M)	F1 (M)
BoW Unigram	Logistic Regression	316	80	11,665	**0.9875**	**0.9878**	**0.9875**	**0.9875**
BoW Unigram	Linear SVC	316	80	11,665	**0.9875**	**0.9878**	**0.9875**	**0.9875**
BoW Unigram	Random Forest	316	80	11,665	**0.9875**	**0.9878**	**0.9875**	**0.9875**
TF-IDF Unigram	Logistic Regression	316	80	11,665	0.9500	0.9545	0.9500	0.9499
TF-IDF Unigram	Linear SVC	316	80	11,665	**0.9875**	**0.9878**	**0.9875**	**0.9875**
TF-IDF Unigram	Random Forest	316	80	11,665	**0.9875**	**0.9878**	**0.9875**	**0.9875**
TF-IDF 1–3-gram	Logistic Regression	316	80	177,913	0.9500	0.9545	0.9500	0.9499
TF-IDF 1–3-gram	Linear SVC	316	80	177,913	0.9375	0.9444	0.9375	0.9373
TF-IDF 1–3-gram	Random Forest	316	80	177,913	**0.9875**	**0.9878**	**0.9875**	**0.9875**

7.2 Accuracy Comparison

Figure 3 plots the test accuracy of all nine configurations side by side.

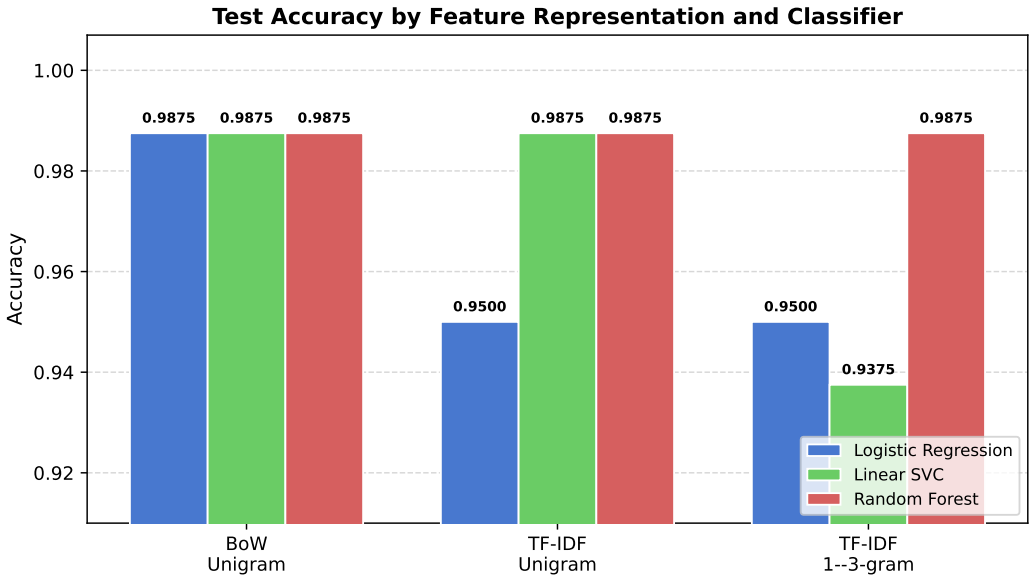

Figure 3: Test accuracy grouped by feature representation. Random Forest achieves 98.75% in all three feature settings.

7.3 F1-Score Heat-map

Figure 4 visualises the macro F1-score across all configurations as a heat-map, making it easy to identify which classifier–feature combination is weakest.

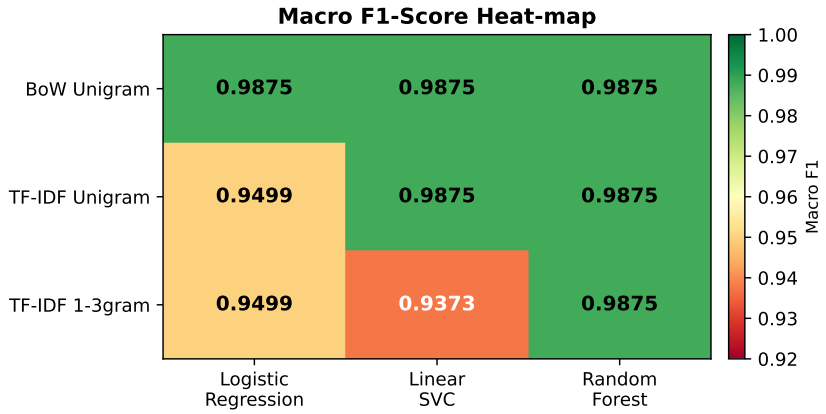

Figure 4: Macro F1-score heat-map. Green = higher is better. The weakest configuration is LinearSVC on TF-IDF 1–3-gram (F1 = 0.9373).

7.4 Key Observations

1. Random Forest is the most robust classifier. Random Forest achieves 98.75% accuracy (F1 = 0.9875) on *all three* feature representations. Its ensemble of 200 decision trees effectively handles both the 11,665-feature unigram space and the 177,913-feature n-gram space without performance degradation.

2. BoW outperforms TF-IDF for Logistic Regression. Logistic Regression drops from 98.75% (BoW) to 95.00% (TF-IDF unigram and TF-IDF 1–3-gram). This is counter-intuitive since TF-IDF is generally preferred, but can occur when: (a) the dataset is small enough that raw count magnitudes are already discriminative, and (b) TF-IDF normalisation suppresses the absolute frequency signal that Logistic Regression uses to set its decision boundary.

3. LinearSVC degrades on the n-gram feature space. LinearSVC falls from 98.75% (BoW and TF-IDF unigram) to 93.75% (TF-IDF 1–3-gram). The dramatic increase in feature dimensionality (11,665 $\rightarrow$ 177,913) introduces many correlated and redundant n-gram features that make the SVM margin harder to optimise with a fixed iteration budget (`max_iter=5000`).

4. Overall task difficulty. Even the weakest configuration achieves 93.75% accuracy on a balanced binary classification task with 80 test samples. Sports and Politics articles have very distinct vocabularies (team names, scores, and sport-specific verbs vs. government terminology, political parties, and policy language), making this a comparatively easy separation task.

7.5 Summary of Best Configurations

Table 4: Top-3 configurations ranked by accuracy.

Rank	Configuration	Accuracy	F1 (macro)
1 (tie)	BoW Unigram + Random Forest	0.9875	0.9875
1 (tie)	BoW Unigram + Logistic Regression	0.9875	0.9875
1 (tie)	BoW Unigram + Linear SVC	0.9875	0.9875
1 (tie)	TF-IDF Unigram + Linear SVC	0.9875	0.9875
1 (tie)	TF-IDF 1-3gram + Random Forest	0.9875	0.9875
6	TF-IDF Unigram + Logistic Reg.	0.9500	0.9499
6	TF-IDF 1-3gram + Logistic Reg.	0.9500	0.9499
8	TF-IDF 1-3gram + Linear SVC	0.9375	0.9373

8 Limitations

Despite the high accuracy, the system has several important limitations:

1. Category labelling noise. The Politics class was collected via the *search API* rather than a true category membership, because `Category:Politics` was empty. A search for "politics" returns articles that *contain* the word but may not be primarily political in nature (e.g., a sports article mentioning government funding for stadiums). This introduces label noise in the Politics class.

2. Small dataset. Only 198 articles per class (396 total) were used. The 80-article test set means each percentage point of accuracy corresponds to less than one article. Results on small test sets have high variance. A 5-fold cross-validation would give more reliable estimates.

3. No stop-word removal or stemming. Common function words (*the, of, is*) add noise to the unigram vocabulary and take up feature dimensions without providing discriminative power. Standard NLP pipelines use stop-word removal and stemming (e.g., Porter stemmer) to reduce vocabulary size and improve generalisation.

4. No minimum document-frequency threshold. The `build_vocabulary_and_df()` function (the version used at runtime) includes every token that appears at least once. Rare tokens (appearing in only one article) are unlikely generalisable features and inflate the vocabulary, increasing memory usage and training time without benefiting accuracy.

5. Memory limitations of dense n-gram matrices. The TF-IDF 1–3-gram matrix has shape $(396 \times 177{,}913)$. Stored as a dense `float64` NumPy array this requires approximately $396 \times 177{,}913 \times 8$ bytes ≈ 563 MB of RAM. Scaling to larger corpora would require sparse matrix representations (e.g., `scipy.sparse.csr_matrix`).

6. Binary task only. The classifier is trained exclusively on the Sports vs. Politics binary distinction. It cannot generalise to finer-grained topic categories (e.g., distinguishing football from tennis within Sports) or to multi-class settings without architectural changes.

7. Domain shift. Wikinews articles tend to be short (often 100–300 words) and written in a neutral encyclopaedic style. The classifier would likely degrade when applied to social-media text, blog posts, or articles from sources with different writing styles due to vocabulary shift.

9 Conclusion

This report presented a complete binary text classification pipeline for distinguishing Sports from Politics articles on Wikinews. Key findings are:

- All three feature representations combined with Random Forest achieved the top accuracy of **98.75%**, confirming the robustness of ensemble methods on text classification.
- Bag-of-Words with raw counts performed on par with or better than TF-IDF in several configurations, suggesting that for a topic-discriminating task with very distinct vocabularies, raw counts already provide sufficient signal.
- Adding n-grams ($n \leq 3$) did not improve and occasionally hurt performance (LinearSVC dropped to 93.75%), because the vast increase in correlated features outweighs the benefit of capturing local word-order context for this task.
- All features were implemented from scratch using only NumPy, verifying the mathematical correctness of the BoW, TF, and IDF formulations.

Future work could include: cross-validation for more reliable estimates, sparse matrix representations for scalability, stop-word removal and stemming for cleaner features, and extending to a multi-class setting across all Wikinews topic categories.

References

[1] Joachims, T. (1998). *Text Categorisation with Support Vector Machines: Learning with Many Relevant Features.* ECML 1998.

[2] Salton, G., & Buckley, C. (1988). *Term-Weighting Approaches in Automatic Text Retrieval.* Information Processing & Management, 24(5).

[3] Manning, C.D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval.* Cambridge University Press.

[4] Breiman, L. (2001). *Random Forests.* Machine Learning, 45(1), 5–32.

[5] Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python.* JMLR, 12, 2825–2830.

[6] Wikinews MediaWiki API. https://en.wikinews.org/w/api.php